

Module – Inheritance Hierarchy and Constructor Chaining

What Is Inheritance?

Key Points

- Inheritance allows one class to acquire fields and methods of another class.
- Promotes code reuse, extensibility, and hierarchical organization.
- The `extends` keyword establishes a parent–child (superclass–subclass) relationship.
- Subclass can override or extend behavior of the superclass.

Code Snippet

```
class Animal {
    void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

Dog d = new Dog();
d.eat();    // inherited from Animal
d.bark();  // defined in Dog
```

Exercise 01 – InheritanceBasics.java

Create a simple hierarchy – Vehicle → Car → ElectricCar. Call both inherited and subclass-specific methods.

Inheritance Hierarchy Example

Key Points

- Multiple levels of inheritance form a class hierarchy.
- Each subclass inherits members from all its ancestors.
- The root of every hierarchy is `java.lang.Object`.

Code Snippet

```
class A { }  
class B extends A { }  
class C extends B { }  
  
System.out.println(new C() instanceof A); // true
```

Hierarchy: `Object` $\rightarrow$ `A` $\rightarrow$ `B` $\rightarrow$ `C`

Exercise 02 – HierarchyLevels.java

Create a 3-level inheritance chain and print which members are inherited at each level.

Constructor Chaining Overview

Key Points

- Constructors in an inheritance chain run from parent to child.
- Every subclass constructor must call one of its superclass constructors.
- If not done explicitly, the compiler inserts `super()` (no-arg constructor).
- Ensures all inherited fields are initialized before subclass initialization.

Execution Order

1. Superclass constructor
2. Subclass constructor

Code Snippet

```
class Parent {  
    Parent() { System.out.println("Parent constructor"); }  
}  
class Child extends Parent {  
    Child() { System.out.println("Child constructor"); }  
}  
  
new Child();
```

Output

```
Parent constructor  
Child constructor
```

Exercise 03 – ConstructorChain.java

Create classes A, B, and C and print constructor messages to observe order.

Calling Superclass Constructors Explicitly

Key Points

- Use `super()` to call a specific parent constructor.
- `super()` must be the first statement in a child constructor.
- If parent lacks a no-arg constructor, you must call one that exists.

Code Snippet

```
class Vehicle {
    Vehicle(String type) {
        System.out.println("Vehicle type: " + type);
    }
}
class Car extends Vehicle {
    Car() {
        super("Car");
        System.out.println("Car created");
    }
}
```

Exercise 04 – SuperConstructor.java

Create a parent class with only a parameterized constructor; subclass must invoke it using `super(arguments)`.

this() vs super() in Constructors

Keyword	Calls	Must Be First	Scope
this()	Another constructor in same class	Yes	Within current class
super()	Constructor of parent class	Yes	Across inheritance chain

Code Snippet

```
class A {  
    A(int x) { System.out.println("A constructor: " + x); }  
}  
class B extends A {  
    B() {  
        this(10);  
        System.out.println("B() called");  
    }  
    B(int y) {  
        super(y);  
        System.out.println("B(int) called");  
    }  
}
```

Output

```
A constructor: 10  
B(int) called  
B() called
```

Exercise 05 – ThisVsSuper.java

Experiment with `this()` and `super()` in chained constructors to trace the call sequence.

Multi-Level Constructor Chaining

Key Points

- Constructors are invoked up the hierarchy first.
- Initialization order: Static blocks → Instance initializers → Constructors.

Code Snippet

```
class A { A() { System.out.println("A"); } }  
class B extends A { B() { System.out.println("B"); } }  
class C extends B { C() { System.out.println("C"); } }  
  
new C();
```

Output

```
A  
B  
C
```

Exercise 06 – MultiLevelChain.java

Add print statements in each constructor to verify order across three levels.

Parameter Passing in Constructor Chaining

Key Points

- Arguments can be passed up the chain to initialize parent fields.
- Each class can add its own fields while reusing parent initialization.

Code Snippet

```
class Person {
    String name;
    Person(String name) {
        this.name = name;
        System.out.println("Person: " + name);
    }
}
class Employee extends Person {
    int id;
    Employee(String name, int id) {
        super(name);
        this.id = id;
        System.out.println("Employee ID: " + id);
    }
}
```

Exercise 07 – ParameterizedChain.java

Create a two-level constructor chain that passes parameters at each level.

Chaining with Overloaded Constructors

Key Points

- Each constructor can chain to another via `this()` or `super()`.
- Reduces duplication and ensures consistent initialization.

Code Snippet

```
class Device {
    Device(String model) {
        System.out.println("Device model: " + model);
    }
}
class Phone extends Device {
    Phone() {
        this("Default");
    }
    Phone(String model) {
        super(model);
        System.out.println("Phone ready");
    }
}
```

Output

```
Device model: Default
Phone ready
```

Exercise 08 – OverloadedChain.java

Implement overloaded constructors in both parent and child classes and observe execution order.

Object Class at the Root

Key Points

- All classes implicitly extend `Object`.
- Even without `super()`, constructor chaining reaches `Object()` constructor.
- Ensures base methods (`toString()`, `equals()`, `hashCode()`) are available.

Code Snippet

```
class Base { Base() { System.out.println("Base"); } }  
class Derived extends Base { Derived() { System.out.println("Derived"); } }  
  
new Derived(); // internally calls Object() → Base() → Derived()
```

Exercise 09 – ObjectRoot.java

Print constructor messages including an overridden `toString()` to confirm the implicit call to `Object`.

Common Mistakes in Constructor Chaining

- Using both `this()` and `super()` in the same constructor → compilation error
- Forgetting to call `super(args)` when parent has no default constructor
- Placing code before `this()` or `super()` → illegal statement
- Calling overridden methods from constructors (child not initialized yet)

Incorrect Example

```
class Demo extends Base {
    Demo() {
        System.out.println("Before super()"); // invalid
        super();
    }
}
```

Exercise 10 – CommonMistakes.java

Write invalid chaining scenarios and fix them to compile successfully.

Best Practices

- Always initialize base classes first via `super()`.
- Use `this()` to reuse constructors in the same class.
- Keep constructors focused on initialization only.
- Avoid deep inheritance chains (beyond 2–3 levels).
- Prefer composition over complex inheritance.

Exercise 11 – BestPractices.java

Refactor a 3-level class hierarchy to simplify constructor logic using `super()` and `this()` correctly.

Key Takeaways

- Inheritance lets subclasses reuse and extend behavior.
- Constructors execute top-down — from parent to child.
- `super()` ensures parent initialization; `this()` chains within the same class.
- Each constructor must complete its initialization before passing control.
- Constructor chaining makes object creation predictable and safe.

Module – Method Overriding Rules, Covariant Returns, and @Override Annotation

What Is Method Overriding?

Key Points

- Method overriding allows a subclass to provide a new implementation of a method defined in its parent class.
- The method signature (name + parameters) must match exactly.
- At runtime, the subclass version executes — enabling dynamic polymorphism.
- Overriding improves flexibility by customizing inherited behavior.

Code Snippet

```
class Animal {
    void sound() { System.out.println("Animal makes sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}

Animal a = new Dog();
a.sound(); // Dog barks
```

Exercise 01 – BasicOverriding.java

Create Shape and Circle classes where draw() is overridden in the subclass.

Rules for Method Overriding

Rule	Description
1	Same method signature – name, parameters, and order must match exactly.
2	Compatible return type – can be same or covariant (subtype of parent return).
3	Access level – cannot be more restrictive; can be same or more permissive.
4	Exception handling – subclass cannot throw broader checked exceptions.
5	Static / final methods – cannot be overridden.
6	Constructors – cannot be overridden.

Code Snippet

```
class Parent {  
    protected Object getValue() throws IOException { return null; }  
}  
class Child extends Parent {  
    @Override  
    public String getValue() { return "Data"; } // wider access, covariant  
    return  
}
```

Exercise 02 – OverridingRules.java

Try violating each rule (e.g., changing access level or adding broader exceptions) and observe compiler errors.

Access Modifier Rules

Key Points

- Access level cannot be reduced in the subclass.
- Allowed transitions:
 - protected → public ✓
 - public → public ✓
 - protected → protected ✓
 - public → protected ✗
- Private methods are not inherited and cannot be overridden.

Code Snippet

```
class A {  
    protected void show() {}  
}  
class B extends A {  
    @Override  
    public void show() {} // broader access allowed  
}
```

Exercise 03 – AccessModifiers.java

Create a parent and child with varying access modifiers — note which ones compile.

Exception Handling in Overriding

Key Points

- Overriding method cannot declare broader checked exceptions than parent.
- It may:
 - Declare fewer or narrower checked exceptions.
 - Declare any unchecked exceptions.
 - Not declare new or broader checked exceptions.

Code Snippet

```
class Base {  
    void process() throws IOException {}  
}  
class Derived extends Base {  
    @Override  
    void process() throws FileNotFoundException {} // narrower  
    // void process() throws Exception {} // broader, invalid  
}
```

Exercise 04 – ExceptionRules.java

Test overriding with both broader and narrower checked exceptions.

Covariant Return Types

Key Points

- Covariant returns (introduced in Java 5) allow overridden methods to return subtypes of the parent's return type.
- Improves flexibility and eliminates type casting.

Code Snippet

```
class Parent {  
    Parent getInstance() { return new Parent(); }  
}  
class Child extends Parent {  
    @Override  
    Child getInstance() { return new Child(); } // covariant return  
}
```

Behavior

Covariant return allows calling subclass methods without casting.

Exercise 05 – CovariantReturn.java

Create a base and derived class where the overriding method returns a more specific subclass type.

Method Resolution at Runtime (Dynamic Dispatch)

Key Points

- Method calls are resolved at runtime based on the object type, not the reference type.
- Compile-time type checking ensures the method exists.
- Enables true runtime polymorphism.

Code Snippet

```
Animal a = new Dog(); // reference → Animal, object → Dog  
a.sound(); // Dog's version executes
```

Exercise 06 – DynamicDispatch.java

Create multiple subclasses overriding a base method. Call them through base references and observe runtime behavior.

Using @Override Annotation

Key Points

- @Override indicates a method overrides one from a parent class.
- Provides compile-time checking — prevents typos and signature mismatches.
- Best practice: use @Override on all overridden methods.

Code Snippet

```
class A {
    void display() {}
}
class B extends A {
    @Override
    void display() { System.out.println("Overridden!"); }

    // @Override
    // void display() {} // compile error: method does not override
}
```

Exercise 07 – OverrideAnnotation.java

Write a method without @Override, misspell its name, then add @Override to see compiler validation.

Static and Final Methods (Not Overridable)

Key Points

- static methods are hidden, not overridden.
- final methods cannot be overridden.
- Hidden static methods depend on reference type, not object type.

Code Snippet

```
class Base {
    static void greet() { System.out.println("Base"); }
    final void info() { System.out.println("Final"); }
}
class Derived extends Base {
    static void greet() { System.out.println("Derived"); } // hides
    // void info() {} // cannot override final method
}

Base b = new Derived();
b.greet(); // Base
```

Exercise 08 – StaticFinalMethods.java

Create static and final methods in a superclass and observe hiding vs overriding behavior.

Overriding vs Overloading

Feature	Overriding	Overloading
Relationship	Between superclass and subclass	Within same class
Signature	Same name & parameters	Same name, different parameters
Return Type	Covariant allowed	Can vary freely
Binding	Runtime	Compile-time
Purpose	Extend or modify inherited behavior	Add variations of same behavior

Code Snippet

```
class Example {  
    void show(int x) {} // overload  
}  
class Derived extends Example {  
    @Override  
    void show(int x) {} // override  
}
```

Exercise 09 – OverloadVsOverride.java

Demonstrate both overloading and overriding and compare compiler vs runtime resolution.

Calling Superclass Method (super.method())

Key Points

- Use super.method() to call the parent's version of an overridden method.
- Useful when extending parent logic instead of replacing it.

Code Snippet

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Cat extends Animal {
    @Override
    void sound() {
        super.sound();
        System.out.println("Cat meows");
    }
}
```

Exercise 10 – SuperMethodCall.java

Override a method and use super.method() to combine parent and child behavior.

Common Mistakes in Overriding

- Method name or parameters mismatch (not true overriding).
- Forgetting `@Override` — no compiler error, wrong behavior.
- Throwing broader checked exceptions.
- Declaring overridden method static or final.
- Reducing access level (e.g., public → protected).

Incorrect Example

```
class Parent {  
    void greet() {}  
}  
class Child extends Parent {  
    void greet(String msg) {} // overloads, not overrides  
}
```

Exercise 11 – CommonMistakes.java

Write incorrect override examples and fix them by applying correct signatures and annotations.

Best Practices

- Always use `@Override` to prevent signature mismatches.
- Keep overridden behavior consistent with superclass intent.
- Use `super.method()` when extending, not replacing logic.
- Follow the Liskov Substitution Principle (LSP).
- Document behavioral differences clearly when overriding.

Exercise 12 – BestPractices.java

Override methods in a parent-child hierarchy, use `super()` where appropriate, and verify LSP compliance.

Key Takeaways

- Method overriding enables runtime polymorphism in Java.
- Follow strict rules for signature, access, and exceptions.
- Covariant returns improve flexibility and safety.
- `@Override` ensures correctness at compile-time.
- Avoid overriding static or final methods.
- Consistent, clean overrides make class hierarchies maintainable and robust.

Module – Abstract Classes and Abstract Methods

What Are Abstract Classes?

Key Points

- An abstract class is declared using the `abstract` keyword.
- It can contain both implemented and unimplemented (abstract) methods.
- Used when you want to provide a common base but prevent direct instantiation.
- Serves as a template for subclasses to build upon.

Code Snippet

```
abstract class Shape {  
    abstract double area(); // abstract method  
    void display() { System.out.println("Shape details"); } // concrete  
method  
}
```

Exercise 01 – AbstractClassIntro.java

Create an abstract class Shape with `area()` as abstract and `display()` as concrete.

Characteristics of Abstract Classes

Feature	Description
Keyword	Declared using abstract
Instantiation	Cannot be instantiated directly
Methods	May contain abstract and non-abstract methods
Constructors	Allowed, but called through subclass constructors
Fields	Can have both static and instance variables
Purpose	To enforce a common structure while allowing subclass specialization

Code Snippet

```
abstract class Animal {  
    String name;  
    Animal(String name) { this.name = name; }  
}
```

Exercise 02 – AbstractClassProps.java

Add variables and constructors to an abstract class and see how they behave when inherited.

What Are Abstract Methods?

Key Points

- Declared with `abstract` and have no method body.
- Forces subclasses to provide their own implementation.
- Can exist only inside abstract classes or interfaces.
- End with a semicolon instead of braces.

Code Snippet

```
abstract class Vehicle {  
    abstract void start(); // abstract method  
}
```

Exercise 03 – AbstractMethod.java

Create an abstract class `Appliance` with abstract method `turnOn()` and implement it in subclasses.

Subclass Implementation

Key Points

- A concrete subclass must implement all inherited abstract methods.
- If it doesn't, it too becomes abstract.
- Subclasses can override and add more functionality.

Code Snippet

```
abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Exercise 04 – SubclassImplementation.java

Create an abstract Instrument class with play() as abstract, then implement Guitar and Drum.

Instantiation Rules

Key Points

- Abstract classes cannot be instantiated directly.
- You can declare references of an abstract type.
- Constructors in abstract classes run when subclass objects are created.

Code Snippet

```
abstract class Bird {
    Bird() { System.out.println("Bird created"); }
    abstract void fly();
}
class Sparrow extends Bird {
    void fly() { System.out.println("Sparrow flies"); }
}

Bird b = new Sparrow(); // allowed reference
b.fly();
```

Exercise 05 – InstantiationRules.java

Try instantiating an abstract class directly and through a subclass to observe the behavior.

Abstract Class with Constructors

Key Points

- Abstract classes can define constructors for shared initialization logic.
- Called automatically during subclass object creation.
- Ensures consistent base setup across subclasses.

Code Snippet

```
abstract class Employee {
    String name;
    Employee(String name) {
        this.name = name;
        System.out.println("Employee constructor");
    }
}
class Manager extends Employee {
    Manager(String name) { super(name); }
}
```

Exercise 06 – AbstractConstructor.java

Create an abstract base class Device with a constructor and subclass Phone that calls super().

Abstract Class Hierarchies

Key Points

- You can form multi-level abstract hierarchies.
- Intermediate abstract classes can partially implement parent methods.
- The final concrete class must complete all implementations.

Code Snippet

```
abstract class Shape { abstract void draw(); }
abstract class Polygon extends Shape {
    void sides() { System.out.println("Has many sides"); }
}
class Triangle extends Polygon {
    void draw() { System.out.println("Drawing triangle"); }
}
```

Exercise 07 – AbstractHierarchy.java

Create a three-level hierarchy and experiment with partial implementation at the middle level.

Mixing Abstract and Concrete Methods

Key Points

- Abstract classes can include utility or helper methods.
- Useful for sharing logic across subclasses.

Code Snippet

```
abstract class Account {  
    abstract double calculateInterest();  
    void printHeader() {  
        System.out.println("=== Account Summary ===");  
    }  
}
```

Exercise 08 – MixedMethods.java

Implement a concrete subclass that uses both abstract and concrete methods from the base class.

Abstract vs Concrete Classes

Aspect	Abstract Class	Concrete Class
Instantiation	Not possible	Possible
Abstract Methods	May contain	Not allowed
Constructors	Allowed	Allowed
Purpose	Common base or template	Fully functional implementation
Example Use	Base entity like Shape or Vehicle	Actual class like Circle or Car

Exercise 09 – CompareAbstractConcrete.java

Refactor a complete class into an abstract one and delegate behavior to a subclass.

Abstract Class vs Interface

Feature	Abstract Class	Interface
Inheritance	Single	Multiple (implements many)
Fields	Can have instance and static fields	Only public static final constants
Methods	Both abstract and concrete allowed	Only abstract (default/static allowed since Java 8)
Constructors	Yes	No
Use Case	Shared state and partial implementation	Defining capabilities or contracts

Code Snippet

```
abstract class Animal { abstract void move(); }
interface Flyable { void fly(); }

class Bird extends Animal implements Flyable {
    void move() { System.out.println("Bird moves"); }
    public void fly() { System.out.println("Bird flies"); }
}
```

Exercise 10 – AbstractVsInterface.java

Implement both an abstract class and an interface in one class to see how they complement each other.

Anonymous Subclasses of Abstract Classes

Key Points

- You can create an anonymous inner class to provide immediate implementation.
- Useful for one-time use or quick testing.

Code Snippet

```
abstract class Task { abstract void execute(); }

Task t = new Task() {
    void execute() { System.out.println("Running anonymously!"); }
};
t.execute();
```

Exercise 11 – AnonymousAbstract.java

Create an anonymous subclass of an abstract class and implement its abstract method inline.

Common Mistakes

- Declaring abstract methods in non-abstract classes.
- Forgetting to implement all abstract methods in subclass.
- Trying to instantiate abstract classes directly.
- Using abstract with final or static methods.

Incorrect Example

```
abstract class Demo {  
    static abstract void show(); // cannot be both static and abstract  
}
```

Exercise 12 – CommonMistakes.java

Write invalid abstract class declarations and fix them so they compile.

Best Practices

- Use abstract classes for shared structure or behavior across related subclasses.
- Keep them focused and lean — avoid unnecessary logic.
- Prefer interfaces for defining capabilities; use abstract classes for shared code.
- Use protected access for subclass-specific fields or methods.
- Document subclass responsibilities clearly.

Exercise 13 – BestPractices.java

Design an abstract base class `Payment` with abstract method `process()`, then create subclasses like `CreditCardPayment` and `UPIPayment`.

Key Takeaways

- Abstract classes define templates with incomplete behavior.
- Abstract methods enforce subclass implementation.
- Cannot be instantiated but can include constructors and concrete methods.
- Enable partial reuse of logic while maintaining extensibility.
- Often combined with interfaces for modular, clean design.

Module – Sealed Classes and Permitted Subclasses (Java 21)

What Are Sealed Classes?

Key Points

- Sealed classes, introduced in Java 17 and finalized in Java 21, let you control inheritance explicitly.
- They restrict which other classes or interfaces can extend or implement them.
- Used to maintain strict control over type hierarchies and enforce design intent.
- Improve readability, security, and pattern-matching safety.

Code Snippet

```
public sealed class Shape permits Circle, Rectangle, Square {}
```

Meaning: Only Circle, Rectangle, and Square can extend Shape.

Exercise 01 – SealedIntro.java

Create a sealed class Vehicle and permit only Car and Bike subclasses.

Why Sealed Classes?

Key Points

- Prevent uncontrolled subclassing in large APIs.
- Enforce domain-specific hierarchies.
- Make pattern matching and switch expressions exhaustive and safe.
- Reduce security risks from unintended inheritance.

Example Motivation

```
abstract class Shape {} // any class can extend (undesired)
sealed class Shape permits Circle, Rectangle {} // controlled extension
```

Exercise 02 – WhySealed.java

Compare behavior of a normal abstract class vs a sealed class by adding unauthorized subclasses.

Permitted Subclasses

Key Points

- The permits clause lists all allowed subclasses.
- Each subclass must be declared in the same module or package.
- If all permitted subclasses are in one source file, the permits clause can be omitted.

Code Snippet

```
public sealed class Shape permits Circle, Square, Rectangle {}

final class Circle extends Shape {}
non-sealed class Square extends Shape {}
sealed class Rectangle extends Shape permits FilledRectangle {}
final class FilledRectangle extends Rectangle {}
```

Exercise 03 – PermittedSubclasses.java

Create a multi-level hierarchy with both sealed and non-sealed subclasses.

Keywords: sealed, non-sealed, and final

Keyword Meaning

sealed Restricts subclassing to explicitly permitted types

non-sealed Lifts restriction, allowing further subclassing

final Prevents inheritance completely

Example

```
sealed class Vehicle permits Car, Truck {}  
final class Car extends Vehicle {}           // no further subclassing  
non-sealed class Truck extends Vehicle {}    // open for more subclasses
```

Exercise 04 – SealedKeywords.java

Create one sealed, one final, and one non-sealed subclass. Try extending each and note compiler behavior.

Rules for Sealed Classes

Rule Description

- 1 Must declare permitted subclasses using permits.
- 2 All permitted subclasses must directly extend or implement the sealed type.
- 3 Each permitted subclass must be final, sealed, or non-sealed.
- 4 Permitted subclasses must reside in the same module or package.
- 5 Compiler verifies completeness — no missing subclasses allowed.

Exercise 05 – SealedRules.java

Attempt to create an unauthorized subclass or omit modifiers and observe compiler errors.

Example: Complete Sealed Hierarchy

Code Snippet

```
sealed interface Shape permits Circle, Rectangle {}

final class Circle implements Shape {
    double radius;
    Circle(double r) { this.radius = r; }
}

non-sealed class Rectangle implements Shape {
    double width, height;
}
```

Output

Only Circle and Rectangle can implement Shape.

Rectangle can have more subclasses; Circle cannot.

Exercise 06 – SealedInterface.java

Define a sealed interface and permit multiple implementing classes with different modifiers.

Sealed Interfaces

Key Points

- Sealed applies to interfaces as well as classes.
- Defines restricted polymorphism for specific capabilities.
- All permitted types must implement the interface directly.

Code Snippet

```
sealed interface Operation permits Add, Subtract, Multiply {}

final class Add implements Operation {}
final class Subtract implements Operation {}
final class Multiply implements Operation {}
```

Exercise 07 – SealedInterfaceOps.java

Create a sealed interface PaymentMethod and permit only UPI, Card, and Cash.

Sealed and Pattern Matching (Java 21)

Key Points

- Sealed hierarchies make pattern-matching switches exhaustive.
- Eliminate the need for a default branch.

Code Snippet

```
sealed interface Shape permits Circle, Rectangle {}

record Circle(double r) implements Shape {}
record Rectangle(double w, double h) implements Shape {}

static double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Rectangle r -> r.w() * r.h();
    };
}
```

Exercise 08 – SealedPatternMatch.java

Use a sealed interface and pattern-matching switch to compute area for multiple shapes.

Combining Sealed with Records

Key Points

- Records are ideal for sealed hierarchies since they are final and immutable.
- Sealed + Record ensures a safe and concise closed data model.

Code Snippet

```
sealed interface Shape permits Circle, Rectangle {}  
record Circle(double radius) implements Shape {}  
record Rectangle(double width, double height) implements Shape {}
```

Advantages

- No mutable state
- Exhaustive pattern matching
- Minimal boilerplate

Exercise 09 – SealedWithRecord.java

Implement a sealed interface Vehicle and define Car and Bike as records.

Sealed vs Abstract vs Final

Type	Inheritance Control	Can Contain Abstract Methods	Extendable Outside Module
abstract	Open	Yes	Yes
final	Closed	No	No
sealed	Restricted	Yes	No (only permitted types)

Exercise 10 – SealedVsOthers.java

Create one hierarchy each using abstract, final, and sealed; compare their extension flexibility.

Practical Use Cases

- Domain Modeling – Payment → UPI, Card, Cash.
- Event Systems – Event → LoginEvent, LogoutEvent.
- AST/DSL Design – Restrict valid node types.
- Security/APIs – Prevent unauthorized subclassing.

Code Snippet

```
sealed class Payment permits UPI, Card, Cash {}  
final class UPI extends Payment {}  
final class Card extends Payment {}  
final class Cash extends Payment {}
```

Exercise 11 – UseCases.java

Model a sealed hierarchy for Document → PDF, Word, Text.

Common Mistakes

- Missing or mismatched permitted subclass declarations.
- Forgetting to mark subclasses as final, sealed, or non-sealed.
- Placing subclasses in a different package or module.
- Using permits without defining all listed subclasses.

Incorrect Example

```
sealed class Shape permits Circle {} // Rectangle exists but not listed
final class Rectangle extends Shape {}
```

Exercise 12 – CommonMistakes.java

Introduce subclass configuration errors and fix them to compile successfully.

Best Practices

- Keep sealed hierarchies concise and well-defined.
- Prefer sealed interfaces with records for clear domain modeling.
- Document all permitted subclasses.
- Use non-sealed selectively to open specific branches.
- Combine with pattern matching for safety and readability.

Exercise 13 – BestPractices.java

Design a sealed interface Notification with EmailNotification, SMSNotification, and a non-sealed PushNotification.

Key Takeaways

- Sealed classes restrict subclassing to a known, controlled set.
- Enhance design clarity, type safety, and pattern-matching integration.
- Subclasses must be final, sealed, or non-sealed.
- Enable closed hierarchies ideal for fixed domain models.
- Work seamlessly with records and pattern matching in Java 21.

Module – instanceof and Pattern Matching for Type Checks (Java 21)

The Role of Type Checking

Key Points

- Type checks determine whether an object is an instance of a given class or interface.
- Traditionally, developers used `instanceof` followed by explicit casting.
- From Java 16 onwards, pattern matching simplifies this by combining the check and cast in one step.

Example (Traditional)

```
if (obj instanceof String) {  
    String s = (String) obj; // explicit cast  
    System.out.println(s.toUpperCase());  
}
```

Exercise 01 – TraditionalTypeCheck.java

Use the old `instanceof` + cast pattern for two or three different object types.

Pattern Matching Simplifies Type Checks

Key Points

- Pattern matching introduces type test and variable binding in one step.
- No explicit casting needed — compiler safely infers the type.
- Syntax: `if (obj instanceof Type varName) { ... }`

Code Snippet

```
Object obj = "Hello Java 21";

if (obj instanceof String s) {
    System.out.println(s.toUpperCase());
}
```

Exercise 02 – PatternMatchingIntro.java

Rewrite a legacy `instanceof` + `cast` example using pattern matching syntax.

Flow Scoping and Smart Casting

Key Points

- The variable introduced (s in the example) is in scope only within the true branch.
- Compiler performs flow analysis to ensure variable access is valid.
- If the condition is false, the pattern variable goes out of scope.

Code Snippet

```
Object obj = "Pattern Matching";

if (obj instanceof String s && s.length() > 5) {
    System.out.println("Valid string: " + s);
}
```

Exercise 03 – FlowScope.java

Try printing the pattern variable outside the if block and observe the compiler error.

Negated Type Checks

Key Points

- Pattern variables are not in scope after negated conditions (`!instanceof`).
- This prevents unsafe access to uninitialized variables.

Code Snippet

```
if (!(obj instanceof String s)) {  
    System.out.println("Not a String");  
} else {  
    System.out.println(s.toUpperCase()); // valid here  
}
```

Exercise 04 – NegatedCheck.java

Write a condition with negated pattern matching and observe scope behavior.

Pattern Matching with Logical Operators

Key Points

- Pattern variables work with logical AND (&&) and OR (||) operators.
- Compiler promotes pattern variable visibility only when definitely matched.

Code Snippet

```
if (obj instanceof String s && s.startsWith("J")) {  
    System.out.println("Starts with J: " + s);  
}
```

Exercise 05 – LogicalPattern.java

Combine instanceof with logical operators to add extra conditions on matched types.

Example: Mixed Type Handling

Code Snippet

```
Object data = 42;
```

```
if (data instanceof String s) {  
    System.out.println("String: " + s);  
} else if (data instanceof Integer i) {  
    System.out.println("Integer doubled: " + (i * 2));  
} else {  
    System.out.println("Unknown type");  
}
```

Output

Integer doubled: 84

Exercise 06 – MultiTypeCheck.java

Write a multi-type handler using pattern matching for String, Integer, and Double.

Pattern Matching in Switch (Java 21)

Key Points

- switch now supports type patterns — no need for manual casting.
- Makes control flow concise and type-safe.

Code Snippet

```
static String formatData(Object obj) {  
    return switch (obj) {  
        case Integer i -> "Integer: " + (i * i);  
        case String s   -> "String: " + s.toUpperCase();  
        case null       -> "Null value";  
        default         -> "Unknown type";  
    };  
}
```

Exercise 07 – SwitchPattern.java

Implement a switch expression using pattern matching for multiple object types.

Guarded Patterns (when clause)

Key Points

- Java 21 introduces guarded patterns — adding extra conditions inside a case.
- Syntax: `case Type var when (condition) -> ...`
- Improves readability and avoids nested if checks inside switch cases.

Code Snippet

```
static String classify(Object o) {  
    return switch (o) {  
        case String s when s.length() > 5 -> "Long String";  
        case String s -> "Short String";  
        case Integer i when i > 100 -> "Large Number";  
        default -> "Other";  
    };  
}
```

Exercise 08 – GuardedPatterns.java

Create a method using a switch with guarded cases to classify objects based on both type and value.

Pattern Matching with Records

Key Points

- Records with pattern matching allow compact, destructured type checks.
- Enables deconstruction patterns inside switch or if statements.

Code Snippet

```
record Point(int x, int y) {}

static void print(Object obj) {
    if (obj instanceof Point(int x, int y)) {
        System.out.println("Coordinates: " + x + ", " + y);
    }
}
```

Exercise 09 – RecordPattern.java

Create a Person(String name, int age) record and use pattern matching to print name and age if matched.

Type Safety and Exhaustiveness

Key Points

- Compiler ensures safe variable binding — no invalid casts.
- When used with sealed hierarchies, switch becomes exhaustive (no default needed).
- Unmatched types trigger compile-time error in sealed contexts.

Code Snippet

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}

static double area(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.r() * c.r();
        case Square s -> s.s() * s.s();
    };
}
```

Exercise 10 – SealedSwitch.java

Combine a sealed interface with pattern matching switch to ensure exhaustive case handling.

Common Mistakes

- Using pattern variables outside their scope.
- Overlapping case labels in switch (compiler error).
- Forgetting to handle null (switch throws NullPointerException if not handled).
- Declaring pattern variables with the same name in nested scopes.

Incorrect Example

```
if (obj instanceof String s)
    System.out.println(s);
System.out.println(s); // not in scope
```

Exercise 11 – CommonMistakes.java

Write an invalid pattern scope example, observe compiler feedback, and fix it.

Best Practices

- Prefer pattern matching for cleaner, safer type checks.
- Always include a null case in switch expressions.
- Combine with sealed classes for exhaustive compile-time safety.
- Use meaningful variable names for pattern bindings.
- Avoid nested type checks — prefer switch-based pattern matching.

Exercise 12 – BestPractices.java

Refactor a legacy type-checking function using modern pattern matching and add null handling.

Key Takeaways

- Pattern matching merges type testing and casting into one safe construct.
- Removes redundancy and minimizes casting errors.
- Works seamlessly in if-else, switch, and sealed hierarchies.
- Supports guarded patterns and record deconstruction in Java 21.
- Encourages cleaner, more declarative code for polymorphic type handling.

Module – Casting Between Reference Types and ClassCastException Scenarios

What Is Type Casting?

Key Points

- Type casting converts one reference type into another related type.
- Commonly used in inheritance hierarchies where superclass and subclass objects interact.
- Enables access to subclass features using superclass references (via downcasting).

Code Snippet

```
Animal a = new Dog(); // upcasting  
Dog d = (Dog) a;      // downcasting
```

Terminology

- Upcasting: Subclass → Superclass (safe, implicit)
- Downcasting: Superclass → Subclass (explicit, may fail)

Exercise 01 – TypeCastingIntro.java

Create a base class Vehicle and subclass Car; perform both upcasting and downcasting.

Upcasting (Safe Cast)

Key Points

- Converts a subclass reference to a superclass type.
- Implicitly handled by the compiler and always safe.
- Subclass-specific methods are no longer accessible through the upcast reference.

Code Snippet

```
Dog dog = new Dog();
Animal animal = dog; // Upcast (implicit)
animal.eat();         // works
// animal.bark();    // not accessible
```

Exercise 02 – Upcasting.java

Demonstrate which methods remain accessible after upcasting.

Downcasting (Explicit Cast)

Key Points

- Used to access subclass-specific methods from a superclass reference.
- Requires an explicit cast.
- Valid only when the object's actual type matches the target subclass.

Code Snippet

```
Animal a = new Dog();    // upcast
Dog d = (Dog) a;         // downcast (valid)
d.bark();                // works fine
```

Exercise 03 – Downcasting.java

Use a superclass reference holding different subclass objects and perform safe downcasts.

Invalid Downcasting (Leads to ClassCastException)

Key Points

- Occurs when the actual object type doesn't match the cast type.
- The compiler allows it (types are compatible structurally), but it fails at runtime.

Code Snippet

```
Animal a = new Cat();  
Dog d = (Dog) a; // Compiles, but throws ClassCastException at runtime
```

Output

```
Exception in thread "main" java.lang.ClassCastException: Cat cannot be cast  
to Dog
```

Exercise 04 – InvalidCast.java

Try casting an object to an incompatible subclass and observe the runtime exception.

Using instanceof Before Casting

Key Points

- Always verify type compatibility with instanceof before performing a cast.
- Prevents ClassCastException at runtime.

Code Snippet

```
Animal a = new Cat();

if (a instanceof Dog d) {    // pattern matching syntax
    d.bark();
} else {
    System.out.println("Not a Dog!");
}
```

Exercise 05 – InstanceofCheck.java

Use instanceof or pattern matching to protect against invalid downcasts.

Object Hierarchy Example

Code Snippet

```
class Animal { void eat() {} }  
class Dog extends Animal { void bark() {} }  
class Cat extends Animal { void meow() {} }
```

```
Animal a = new Dog();  
((Dog)a).bark();    // OK  
((Cat)a).meow();    // Runtime error
```

Hierarchy

```
Object  
├── Animal  
│   ├── Dog  
│   └── Cat
```

Exercise 06 – CastHierarchy.java

Build a 3-level inheritance hierarchy and test valid and invalid casts.

ClassCastException Deep Dive

Key Points

- Thrown by the JVM when an invalid runtime cast occurs.
- Checked at runtime based on the object's actual class.
- Commonly occurs in collections and polymorphic references.

Common Causes

1. Wrong assumption about an object's actual type.
2. Unsafe casting from generic collections.
3. Misuse of polymorphism or factories.

Example

```
Object obj = "Java";  
Integer num = (Integer) obj; // ClassCastException
```

Exercise 07 – ClassCastDeepDive.java

Simulate multiple invalid casts and handle them using try-catch.

Catching ClassCastException

Key Points

- Can be caught, but it's better to prevent it using design or instanceof.
- Typically indicates a logic flaw in handling object types.

Code Snippet

```
try {
    Object obj = "Java 21";
    Integer num = (Integer) obj;
} catch (ClassCastException e) {
    System.out.println("Invalid cast: " + e.getMessage());
}
```

Exercise 08 – HandleException.java

Wrap a cast in a try-catch block to handle runtime casting errors safely.

Casting and Generics

Key Points

- Type erasure removes generic type info at runtime — unchecked casts may occur.
- Can cause `ClassCastException` when mixing raw and parameterized types.

Code Snippet

```
List list = new ArrayList<String>();  
list.add(100); // raw type allows this  
String s = (String) list.get(0); // ClassCastException
```

Exercise 09 – GenericCast.java

Demonstrate unsafe casting with raw vs parameterized lists.

Safe Design Alternatives

- Use instanceof or pattern matching before casting.
- Prefer polymorphism over explicit casting.
- Use generics correctly for compile-time safety.
- Avoid deep hierarchies that force repeated downcasts.

Code Snippet

```
Animal a = new Dog();  
a.makeSound(); // polymorphism > casting
```

Exercise 10 – SafeDesign.java

Refactor casting-heavy code using polymorphism to eliminate explicit casts.

Common Mistakes

- Casting unrelated classes within the same hierarchy.
- Forgetting to check type before downcasting.
- Mixing raw and generic types.
- Assuming the compiler detects all invalid casts (runtime only).

Incorrect Example

```
Object o = new Object();  
String s = (String) o; // Compiles but fails at runtime
```

Exercise 11 – CommonMistakes.java

Write invalid reference casts and identify compile-time vs runtime errors.

Best Practices

- Minimize explicit casting — use interface-based design.
- Always validate object types before downcasting.
- Use pattern matching for cleaner, safer casting.
- Handle `ClassCastException` gracefully if unavoidable.
- Keep inheritance shallow and logical.

Exercise 12 – BestPractices.java

Apply best practices in a Shape hierarchy (Shape, Circle, Square) ensuring safe casts.

Key Takeaways

- Upcasting is implicit and safe.
- Downcasting is explicit and risky.
- `ClassCastException` occurs at runtime on invalid casts.
- Use `instanceof` or pattern matching to prevent errors.
- Prefer polymorphism and interfaces to avoid excessive casting.

Module – Interfaces: Abstract, Default, Static, and Private Methods (Java 21)

What Is an Interface?

Key Points

- An interface defines a contract that implementing classes must follow.
- Contains method declarations, constants, and from Java 8+, default and static methods.
- All interface members are implicitly:
 - public and abstract (for methods by default)
 - public, static, and final (for variables)

Code Snippet

```
interface Drawable {  
    void draw(); // abstract by default  
}
```

Exercise 01 – InterfaceBasics.java

Create an interface Shape with a method area() and implement it in two classes (Circle, Rectangle).

Abstract Methods in Interfaces

Key Points

- Declared without a body and must be implemented by the class.
- Define the contract that implementing classes must fulfill.
- All implementing classes must define these methods unless they are abstract themselves.

Code Snippet

```
interface Animal {  
    void makeSound(); // implicitly public & abstract  
}  
  
class Dog implements Animal {  
    public void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Exercise 02 – AbstractMethods.java

Create an interface Playable with abstract start() and stop() methods; implement it in MusicPlayer class.

Default Methods (Java 8+)

Key Points

- Provide default implementations inside interfaces.
- Allow adding new behavior without breaking existing implementations.
- Declared using the default keyword.

Code Snippet

```
interface Vehicle {  
    default void start() {  
        System.out.println("Starting vehicle...");  
    }  
}  
  
class Car implements Vehicle {}  
  
new Car().start();
```

Output

Starting vehicle...

Exercise 03 – DefaultMethods.java

Create interface Printer with a default print() method and override it in a subclass to customize behavior.

Overriding Default Methods

Key Points

- Implementing classes can override default methods.
- If multiple interfaces provide the same default method, the class must override it to resolve conflicts.

Code Snippet

```
interface A {  
    default void show() { System.out.println("A"); }  
}  
interface B {  
    default void show() { System.out.println("B"); }  
}  
class C implements A, B {  
    public void show() { System.out.println("C resolves conflict"); }  
}
```

Exercise 04 – DefaultConflict.java

Create two interfaces with the same default method and implement both in one class to resolve ambiguity.

Static Methods in Interfaces

Key Points

- Introduced in Java 8.
- Belong to the interface itself and are not inherited by classes.
- Called using the interface name directly.

Code Snippet

```
interface MathUtil {  
    static int square(int x) {  
        return x * x;  
    }  
}  
  
System.out.println(MathUtil.square(5)); // 25
```

Exercise 05 – StaticMethod.java

Create interface Converter with a static toUpper(String s) method and test it in main method.

Private Methods in Interfaces (Java 9+)

Key Points

- Used to share code between default or static methods.
- Not accessible outside the interface.
- Can be static or instance-level.

Code Snippet

```
interface Logger {  
    default void logInfo(String msg) {  
        log(msg, "INFO");  
    }  
    default void logError(String msg) {  
        log(msg, "ERROR");  
    }  
    private void log(String msg, String level) {  
        System.out.println(level + ": " + msg);  
    }  
}
```

Exercise 06 – PrivateMethods.java

Implement a Logger interface using private helper methods for shared logic between default methods.

Static vs Default vs Abstract vs Private

Type	Keyword	Has Body?	Inheritable?	Access	Purpose
Abstract	(implicit)	No	Yes	Public	Must be implemented
Default	default	Yes	Yes	Public	Extend interface behavior
Static	static	Yes	No	Public	Utility or helper methods
Private	private	Yes	No	Private	Internal code reuse

Exercise 07 – MethodTypes.java

Create an interface combining all four method types and identify which can be accessed and overridden.

Interface Variables and Constants

Key Points

- All interface fields are implicitly public, static, and final.
- Must be initialized at declaration.
- Accessed using the interface name.

Code Snippet

```
interface Config {  
    int TIMEOUT = 30; // public static final  
}  
  
System.out.println(Config.TIMEOUT);
```

Exercise 08 – InterfaceConstants.java

Create interface AppSettings with constants and print them from the main class.

Multiple Inheritance via Interfaces

Key Points

- A class can implement multiple interfaces.
- If conflicts occur (same method signatures), the class must override the method.
- Allows flexible, conflict-free multiple inheritance.

Code Snippet

```
interface Flyable { void move(); }
interface Swimmable { void move(); }

class Duck implements Flyable, Swimmable {
    public void move() { System.out.println("Duck swims and flies"); }
}
```

Exercise 09 – MultipleInterfaces.java

Implement two interfaces with the same abstract method and resolve ambiguity in the class.

Interface Inheritance

Key Points

- Interfaces can extend other interfaces.
- A class implementing the sub-interface must implement all inherited abstract methods.

Code Snippet

```
interface Animal {  
    void eat();  
}  
interface Pet extends Animal {  
    void play();  
}  
class Dog implements Pet {  
    public void eat() {}  
    public void play() {}  
}
```

Exercise 10 – InterfaceInheritance.java

Create Readable and Writable interfaces, extend them into FileHandler, and implement it.

Interfaces and Polymorphism

Key Points

- Interfaces enable polymorphism — same interface, different implementations.
- You can hold references to objects of different classes via the interface type.

Code Snippet

```
interface Payment { void pay(); }

class CardPayment implements Payment {
    public void pay() { System.out.println("Paid by card"); }
}
class UpiPayment implements Payment {
    public void pay() { System.out.println("Paid by UPI"); }
}

Payment p = new UpiPayment();
p.pay(); // runtime dispatch
```

Exercise 11 – InterfacePolymorphism.java

Use an interface to invoke methods on different implementing classes through a common reference.

Common Mistakes

- Forgetting public on implementing methods — default access not allowed.
- Defining instance fields (must be static final).
- Attempting to override static or private interface methods.
- Confusing interface constants with class variables.

Incorrect Example

```
interface Demo {  
    int count; // must initialize  
    private void helper(); // abstract methods can't be private  
}
```

Exercise 12 – CommonMistakes.java

Write invalid interface declarations and correct them to understand compiler enforcement.

Best Practices

- Use interfaces to define contracts, not state.
- Prefer default methods to extend API without breaking implementations.
- Keep interfaces small and focused (Interface Segregation Principle).
- Use private helper methods to avoid duplication in default methods.
- Limit static methods to utility use cases only.

Exercise 13 – BestPractices.java

Refactor a large interface into smaller cohesive ones using default and private methods effectively.

Key Takeaways

- Interfaces define contracts that enable polymorphism and flexibility.
- Abstract methods must be implemented; default methods can be reused or overridden.
- Static methods belong to the interface, not instances.
- Private methods help organize internal logic cleanly.
- Combined, these make interfaces powerful, extensible, and backward-compatible in Java 21

Module – Enums: Constants, Fields, Constructors, Methods & Custom Behaviour (Java 21)

What Are Enums?

Key Points

- An enum defines a fixed set of named constants.
- Each constant is an instance of the enum type.
- Enums are type-safe — you cannot assign unrelated values.
- Enums can include fields, constructors, and methods like classes.

Code Snippet

```
enum Direction { NORTH, SOUTH, EAST, WEST }

Direction d = Direction.NORTH;
System.out.println(d); // NORTH
```

Exercise 01 – EnumBasics.java

Create an enum Day with constants for all days and print their names using a loop.

Enum Internals

Key Points

- Each enum constant is a static final instance of the enum.
- `values()` returns all constants as an array.
- `ordinal()` gives the constant's index (starting at 0).
- `name()` returns the declared constant name as a string.

Code Snippet

```
for (Direction dir : Direction.values()) {  
    System.out.println(dir + " ordinal=" + dir.ordinal());  
}
```

Exercise 02 – EnumInternals.java

Print all enum constants with their ordinal values and names.

Enum with Fields and Constructor

Key Points

- Enums can have fields to store data for each constant.
- Constructors are private or package-private and run once per constant.
- Fields are initialized using the constructor.

Code Snippet

```
enum Planet {
    MERCURY(3.30, 2.44), VENUS(4.87, 6.05), EARTH(5.97, 6.37);

    private final double mass;    // in 10^24 kg
    private final double radius;  // in 10^6 m

    Planet(double mass, double radius) {
        this.mass = mass;
        this.radius = radius;
    }

    double surfaceGravity() {
        double G = 6.67300E-11;
        return G * mass * 1e24 / (radius * 1e6 * radius * 1e6);
    }
}
```

Exercise 03 – EnumWithFields.java

Define an enum Currency with fields symbol and country, initialize via constructor, and print details.

Enum Methods and Behavior

Key Points

- Enums can include methods that operate on fields.
- `toString()` can be overridden for customized output.
- Common behavior can be shared, or overridden per constant.

Code Snippet

```
enum Status {  
    NEW, IN_PROGRESS, DONE;  
  
    public boolean isFinal() {  
        return this == DONE;  
    }  
}  
  
System.out.println(Status.NEW.isFinal());    // false  
System.out.println(Status.DONE.isFinal());    // true
```

Exercise 04 – EnumMethods.java

Add a `getDescription()` method to an enum `OrderStatus` returning custom messages for each constant.

Enum with Custom toString()

Key Points

- Overriding toString() makes enums more readable for display or logs.
- Recommended for user-facing outputs.

Code Snippet

```
enum Priority {  
    LOW, MEDIUM, HIGH;  
  
    @Override  
    public String toString() {  
        return name().charAt(0) + name().substring(1).toLowerCase();  
    }  
}  
  
System.out.println(Priority.HIGH); // High
```

Exercise 05 – CustomToString.java

Override toString() in Severity enum to return formatted labels (e.g., “Critical”, “Minor”).

Enums with Abstract Methods (Custom Behavior per Constant)

Key Points

- Each constant can override abstract methods for custom logic.
- Ideal for representing different behaviors under one type.

Code Snippet

```
enum Operation {
    ADD { double apply(double x, double y) { return x + y; } },
    SUBTRACT { double apply(double x, double y) { return x - y; } },
    MULTIPLY { double apply(double x, double y) { return x * y; } },
    DIVIDE { double apply(double x, double y) { return x / y; } };

    abstract double apply(double x, double y);
}

System.out.println(Operation.ADD.apply(3, 4)); // 7.0
```

Exercise 06 – EnumAbstractMethod.java

Implement an enum ShapeType with an abstract area() method for CIRCLE and SQUARE.

Enum Constructors with Parameters

Key Points

- Each constant can pass arguments to the constructor.
- Constructor initializes fields per constant.
- Enums cannot be instantiated using new.

Code Snippet

```
enum LogLevel {  
    INFO(1), WARNING(2), ERROR(3);  
    private final int severity;  
    LogLevel(int level) { this.severity = level; }  
    int getSeverity() { return severity; }  
}
```

Exercise 07 – EnumConstructorParams.java

Create an enum Membership with BASIC, GOLD, and PLATINUM constants, each having an associated price.

Enum Switch Usage

Key Points

- Enums work seamlessly with switch statements.
- Compiler ensures all constants are handled.
- Java 14+ allows concise switch expressions.

Code Snippet

```
enum TrafficLight { RED, YELLOW, GREEN }

static String action(TrafficLight t) {
    return switch (t) {
        case RED -> "STOP";
        case YELLOW -> "READY";
        case GREEN -> "GO";
    };
}
```

Exercise 08 – EnumSwitch.java

Write a switch expression using enum Season returning different seasonal activities.

Enum Switch Patterns (Java 21)

Key Points

- Pattern matching allows switch over sealed or enum-based hierarchies.
- Enables exhaustive, type-safe decision handling.

Code Snippet

```
sealed interface Command permits Start, Stop {}
enum Start implements Command { INSTANCE }
enum Stop implements Command { INSTANCE }

static String handle(Command cmd) {
    return switch (cmd) {
        case Start s -> "System Starting";
        case Stop t -> "System Stopping";
    };
}
```

Exercise 09 – EnumSwitchPattern.java

Use enums with an interface and handle them using a pattern-matching switch.

Custom Behavior per Constant (Overridden Methods)

Key Points

- Each constant can override methods declared in the enum body.
- Functions like defining anonymous subclasses for each constant.

Code Snippet

```
enum Mood {  
    HAPPY { void react() { System.out.println("Smile"); } },  
    SAD { void react() { System.out.println("Cry"); } },  
    ANGRY { void react() { System.out.println("Shout"); } };  
  
    abstract void react();  
}  
  
Mood.SAD.react(); // Cry
```

Exercise 10 – EnumCustomBehavior.java

Create NotificationType enum with EMAIL, SMS, and PUSH constants, each overriding sendMessage().

Enum Implementing Interfaces

Key Points

- Enums can implement interfaces but not extend classes.
- Enables shared polymorphic behavior across multiple enums.

Code Snippet

```
interface Printable { void print(); }

enum DocumentType implements Printable {
    PDF, DOCX;
    public void print() { System.out.println("Printing " + this); }
}
```

Exercise 11 – EnumInterface.java

Create an enum PaymentMethod implementing an interface Payable with a pay() method.

Enum with Static Methods and Lookup

Key Points

- Enums can define static utility or lookup methods.
- Useful for mapping codes or text to constants.

Code Snippet

```
enum Status {  
    NEW, IN_PROGRESS, DONE;  
  
    static Status fromString(String s) {  
        return Status.valueOf(s.toUpperCase());  
    }  
}  
  
System.out.println(Status.fromString("done")); // DONE
```

Exercise 12 – EnumLookup.java

Add a static method `getByCode(int)` in enum `HttpStatus` to find constants by numeric value.

Common Mistakes

- Declaring public constructors (not allowed).
- Trying to instantiate enums using new.
- Using mutable fields (breaks immutability).
- Forgetting the semicolon before methods or fields.

Incorrect Example

```
enum Color { RED, GREEN, BLUE
    void info() {} // Missing semicolon before methods
}
```

Exercise 13 – CommonMistakes.java

Intentionally create syntax errors (missing semicolon, public constructor) and fix them.

Best Practices

- Use enums instead of integer constants for type safety.
- Add fields and constructors for additional metadata.
- Use abstract methods or per-constant overrides for behavior variation.
- Keep enums concise and focused — avoid excessive logic.
- Combine with switch or pattern matching for clarity.

Exercise 14 – BestPractices.java

Model an enum UserRole with role-based behavior (ADMIN, USER, GUEST) using abstract methods.

Key Takeaways

- Enums are specialized classes with fixed instances.
- Support fields, constructors, and methods.
- Enable per-constant behavior through overrides.
- Work seamlessly with switch and pattern matching.
- Provide a type-safe way to represent finite sets of constants in Java.